程序设计与算法语言 II 课程上机实践安排（三）

一、 目的

1. 掌握运算符重载为成员函数的方法
2. 掌握运算符重载为友元函数的方法

二、 提交时间及截止要求：本周六 22：00 之前。

三、 提交形式：

1） 源代码，以学号+日期命名，如有多个文件，在后面加_1，_2 以区别

2） 程序运行截图，以学号+日期命名，如有多个文件，在后面_1，_2 以区别

四、 提交地址：

东大云盘王文博\ 上机作业 0319 文件夹内

五、 内容

1. 分别用成员函数和友元函数为复数类重载部分运算符，两个头文件内容如下：

```
// complex1 学号.h
  class complex1{
     double real;   //实部
     double imag; //虚部
  public:
     complex1(double r = 0.0 ,double i = 0.0 );   //构造函数
     complex1(const complex1 &);   //复制构造函数
     complex1 operator   +( const complex1 &);        //重载+，实现复数的加运算
     complex1 operator   -( const complex1 &);        //重载-，实现复数的减运算
     complex1 operator   *( const complex1 &);        //重载*，实现复数的乘运算
     complex1 operator   /( const complex1 &);        //重载/，实现复数的除法运算
     complex1 &operator +=( const complex1 &);     //重载+=，实现复数的+=运算
     complex1 &operator -=( const complex1 &);      //重载-=，实现复数的-=运算
     complex1 &operator *=( const complex1 &);      //重载*=，实现复数的*=运算
     complex1 &operator /=( const complex1 &);      //重载/=，实现复数的/=运算
     complex1 &operator =( const complex1 &);      //重载赋值=
     complex1 operator ++();                         //重载复数的前缀++运算，实部虚部均加 1
     complex1 operator ++(int);                     //重载复数的后缀++运算，实部虚部均加 1
     bool operator==( const complex1 &) const; //重载==,比较复数是否相等(实部相等且虚部相等)
     bool operator >( const complex1 &) const       //重载>，实现复数的模值的>比较运算
     complex1 operator -() const;                    //重载-，实现复数的符号取反，-(a+bi)=-a-bi
     double getReal() const;         //取实部
     double getImag()const;          //  取虚部
     double modulus() const;       //取模值
     //完善补充<<和>>重载的友元函数声明
  };
//complex2 学号.h：
class complex2{
     double real;   //实部
```

```
    double imag; //虚部
  public:
    complex2(double r = 0.0 ,double i = 0.0 );     //构造函数
    complex2(const complex2 &);     //复制构造函数
    double getReal() const;         //取实部
    double getImag()const;          //  取虚部
    double modulus() const;        //取模值
    complex2 &operator =( const complex2 &);       //重载赋值=
    //完善补充运算符重载友元函数的声明
    //完善补充<<和>>重载友元函数的声明
};
```

1） 在 complex2 学号.h 文件中完善+、-、*、/、+=、-=、*=、/=、==、-(取负)、>以及前置++和后置++的友元函数的声明；

2） 在 complex1 学号.cpp 和 complex2 学号.cpp 中完成如上成员函数和友元函数定义；

3） 分别完成两个类输出运算符<<和输入运算符>>的重载；

4） 在以 学号 0319.cpp 命名的文件中的主函数中进行如下操作：

✧ 声明 4 个复数类 complex1 的对象 A1（32,-20），B1（12,-7），C1， D1；

✧ 声明 4 个复数类 complex2 的对象 A2（32,-20），B2（12,-7），C2， D2；

✧ 完成重载函数 Ctest 的定义，形参分别对应 complex1 和 complex2 类型，在主函数内有如下函数调用语句，分别测试两种情况下重载的运算符：

```
Ctest(A1,B1,C1,D1);
Ctest(A2,B2,C2,D2);
```

✧ 利用重载的>>完成一个复数 Com 的输入，并完成如下该复数与一个实数的运算

```
CC1=Com+10
CC2= -10+Com
```

附： 重载的Ctest函数体内的内容，如下：

```
{
    cout << "A为:\t\t" << A;
    cout << "B为:\t\t" << B;
    cout << "C为:\t\t" << C;
    cout << "D为:\t\t" << D;
    C = A++;          //后置++运算
    cout << "C=A++，  C为:\t" << C;
    cout << "A为：\t\t" << A;

    C = ++A;          //前置++运算
    cout << "C=++A，  C为:\t" << C;
    cout << "            A为:\t" << A;

    cout << "            B为:\t" << B;
    C = A * B;        //乘法运算
    cout << "C=A*B，  C为:\t" << C;

    A *= B;
    cout << "A*=B，   A为:\t" << A;
```

```
C = A / B;
cout << "C=A/B，  C为:\t" << C;

A /= B;
cout << "A/=B，    A为:\t" << A;

D = A + B + C;
cout << "D=A+B+C,D为:\t" << D;

A += B += C;
cout << "A+=B+=C,A为:\t" << A;
cout << "            B为:\t" << B;

D = A - B - C;
cout << "D=A-B-C,D为:\t" << D;

A -= B += C;
cout << "A-=B+=C,A为:\t" << A;
cout << "            B为:\t" << B;

A = B;
cout << "A=B，    A为:\t" << A;

D = A * B * C;
cout << "D=A*B*C,D为:\t" << D;

D = D / B / C;
cout << "D=D/B/C,D为:\t" << D;

A *= B *= C;
cout<<"A*=B*=C,A为:\t" << A;
cout<<"            B为:\t" << B;
cout<<"            C为:\t" << C;

A /= B /= C /= C;
cout<<"A/=B/=C/=C,A为:\t" << A;
cout<<"            B为:\t" << B;
cout<<"            C为:\t" << C;

if(A>C) cout << " A > C " << endl;
else cout<< " A <= C " << endl;

B = C = A;
cout << "B   =C = A,A为:\t" << A;
cout << "            B为:\t" << B;
cout << "            C为:\t" << C;
```

```
    C = -D;
    cout << "C = -D,C为:\t" << C;
    cout << "              D为:\t" << D;

    if (A == B) cout << "A==B" << endl;
    else cout << "A!=B" << endl;
}
```

运行结果示例如下：



